

浮游植物类群/群落结构 层次预测 —— 代码说明

(code/)

基于分层深度学习的藻类群落结构一致性预报。整体流程：**数据预处理** → **基础模型** (多输出·多步·原始尺度) → **层次协调** → **动态权重端到端协调 (DynamicWLS)** → **SHAP 可解释性** → **可视化 (R)** 。

层次结构 (加和约束) : `Algae_Sum = Green_Algae + Cyanobacteria + Diatoms + Cryptophyta`。

1. 核心建模设定 (当前 multi_output 管线)

项目	设定
尺度	原始尺度 (不做 log) , 仅 StandardScaler 标准化; 预测只反标准化、不 expm1
输入特征	13 个原始特征 : 4 底层藻 (自回归) + pH/Turbidity/WT/precipitation/DIN/DIP/TN/TP/NPR; 不含 Algae_Sum (避免信息泄漏)
输出目标	5 个 : Green_Algae、Cyanobacteria、Diatoms、Cryptophyta、 Algae_Sum
预测方式	一个模型直接多步 : 一次前向输出 <code>t+1/t+2/t+3 × 5 目标 = 15 个值</code> (非递归)
基础模型	4 个架构各 1 个模型: LSTM-FC / LSTM-MLP / LSTM-KAN / 纯KAN
窗口	<code>LOOKBACK=30</code> , <code>HORIZONS=[1,2,3]</code> , 滚动窗口
数据划分	7:1:2 (train 70% / val 10% / test 20%, 按时间顺序; 测试集~完整一年)
超参搜索	分步优化 (坐标下降) : 从基线出发, 按"结构/学习率优先"逐个超参数扫描选优
选优标准	跨全部 (<code>horizon</code> , <code>target</code>) 的 平均验证集 nRMSE 最小
评价指标	nRMSE、nMAE、NSE、KGE (逐 (<code>horizon</code> , <code>target</code>) 在原始尺度计算)
随机种子	<code>SEED=42</code> (多种子稳定性见 <code>best_params_multi_seed.py</code>)

同时保留了早期的**单输出**管线 (`single_output_model/` 等 `*_single*` / 不带 `_multi` 的脚本) 用于单输出 vs 多输出对比, **当前主线是 multi_output**。

2. 环境

推荐 conda 环境 **Bayesian** (Python 3.13) 。主要依赖: **torch**、**pandas**、**numpy**、**scikit-learn**、**efficient-kan**、**shap**、**matplotlib/seaborn/statsmodels** (见 [requirements.txt](#)) 。

```
# 1) 激活环境
conda activate Bayesian
# 若 conda 不在 PATH, 可直接用解释器绝对路径, 例如:
# & "C:\Users\<用户名>\anaconda3\envs\Bayesian\python.exe" 脚本.py

# 2) efficient-kan (PyPI 没有, 需从 GitHub 安装; LSTM-KAN / 纯KAN 依赖它)
python -m pip install "git+https://github.com/Blealtan/efficient-kan.git"
```

SHAP 说明: 当前 **Bayesian** (Py3.13) 下 `import shap` 会因预编译 `_cext` wheel 崩溃, SHAP 步骤暂不可直接运行 (代码已就绪)。修复方式: 换一个能正常 `import shap` 的环境 (如 Py3.11/3.12 + `numpy<2.1` + `shap`) 再跑 `SHAP/shap_lstm_kan_multi.py`。

Windows 路径含中文的注意: 运行脚本时建议先 `cd` 进脚本所在目录, 再用文件名运行 (脚本内部用 `__file__` 定位路径, `cwd` 不影响), 避免把中文绝对路径作为命令行参数传给 `python` 时的编码问题。

烟雾测试模式: 所有训练脚本支持环境变量 `MULTI_SMOKE=1` → 极小搜索 + 2 epoch, 仅验证流程与张量形状 (不产出可用结果)。

3. 目录结构

```
code/
├── requirements.txt
├── data_pre/                                # 数据预处理与 EDA
│   ├── raw_data.csv                        # 原始数据 (九龙江库区, 日频)
│   ├── data_pre(7_1_2).py                  # ★当前划分 7:1:2, 生成模型输入
│   ├── data_pre(7_2_1).py                  # 旧划分 7:2:1 (保留)
│   └── raw_data_EDA.py                     # 探索性数据分析
├── base_model/
│   └── data/                                # 模型输入:
train/val/test_model_input.csv (由 data_pre 生成)
│   ├── common/                             # 公共工具 (单/多输出共用)
│   │   ├── data_utils.py                   # 特征列、窗口构造、标准化、反变换
│   │   ├── train_utils.py                  # 训练循环、DataLoader、预测
│   │   ├── metrics_utils.py                # nRMSE/nMAE/NSE/KGE
│   │   └── seed_utils.py                   # 固定随机种子
│   ├── multi_output_model/                 # ★当前主线: 多输出·多步基础模型
│   │   └── models.py                       # 4 架构模型定义 + 常量
├── (TARGET_COLS/HORIZONS)
│   ├── grid_search_multi_output.py         # LSTM-FC/MLP/KAN 分步搜索
│   ├── kan_full_grid_search_multi.py       # 纯 KAN 分步搜索
│   ├── best_params_multi_seed.py           # 最优超参 × 多种子 (箱线图用)
│   └── single_output_model/                # 旧: 单输出 (对比保留)
├── hierarchical_reconciliation/             # 静态层次协调 BU/TD/OLS/WLS/MinT
└── reconcile_multi_lstm_kan_best_params.py  # ★多输出·逐 horizon 协调
```

```

├── reconcile_single_lstm_kan_best_params.py    # 旧: 单输出 (被多输出复用其数学)
├── dynamic_weight_end_to_end_reconciliation/    # 动态权重端到端协调
│   ├── frozen_dynamic_wls_multi.py            # ★多输出·逐 horizon DynamicWLS (冻结基座)
│   ├── frozen_dynamic_wls.py                  # 旧: 单输出 (被多输出复用其组件)
│   └── fine_tune_dynamic_wls.py                # 旧: 不冻结微调版 (未纳入多输出)
├── SHAP/
│   ├── shap_lstm_kan_multi.py                  # ★多输出·逐 (target,horizon) SHAP (待 shap
环境)
│   └── shap_lstm_kan.py                        # 旧: 单输出
└── visualization/                             # R/ggplot2 绘图 (注: 为单输出结果所写, 多输出
结果需按新列名/路径适配)
    ├── grid_search_plots/ multi_seed_boxplots/ reconciliation_accuracy/
    └── reconciliation_plots/ shap_plots/

```

各脚本运行后会在所在目录生成 `*_outputs*/` 结果目录 (见第 5 节)。

4. 快速开始：端到端训练流程

全部用 `Bayesian` 环境运行。下面命令默认已 `conda activate Bayesian`; 若 `conda` 不在 `PATH`, 把 `python` 换成解释器绝对路径。

```

# 第 0 步: 生成 7:1:2 数据 (写入 base_model/data/)
cd code/data_pre
python "data_pre(7_1_2).py"

# 第 1 步: 基础模型分步搜索 (4 个模型) — 必须先跑, 下游依赖其最优模型
cd ../base_model/multi_output_model
python grid_search_multi_output.py          # LSTM-FC / LSTM-MLP / LSTM-KAN (~10-15
min)
python kan_full_grid_search_multi.py        # 纯 KAN

# 第 2 步 (可选): 最优超参 × 10 随机种子, 产出稳定性箱线图所需指标
python best_params_multi_seed.py

# 第 3 步: 层次协调 (BU/TD/OLS/WLS/MinT), 基座=多输出 LSTM-KAN, 逐 horizon
cd ../../hierarchical_reconciliation
python reconcile_multi_lstm_kan_best_params.py

# 第 4 步: 动态权重端到端协调 (frozen DynamicWLS), 逐 horizon, D1→D2=0
cd ../dynamic_weight_end_to_end_reconciliation
python frozen_dynamic_wls_multi.py

# 第 5 步: SHAP (待 shap 环境修复后, 在可用环境运行)
cd ../SHAP
python shap_lstm_kan_multi.py

```

```
# 第 6 步: 可视化 (R 脚本, 需 ggplot2/scales/tidyverse; 多输出新结果需先适配列名)
# Rscript visualization/.../xxx.R
```

仅冒烟验证 (不产出可用结果) : 在第 1/3/4 步命令前设 `$env:MULTI_SMOKE="1"` (PowerShell) 。

重跑提醒: 分步搜索每次会从头重算并向 `search/*.csv` 追加记录、覆盖 `best_params/best_models`。若要干净重跑, 先删除对应的 `*_outputs*/` 目录。

5. 各阶段功能与产物

数据预处理 `data_pre/`

- `data_pre(7_1_2).py`: 读 `raw_data.csv` → 重命名/构造 `Algae_Sum` → 删除不用变量 → 生成 `log_*` (仅留作可选, 当前多输出不使用) → 按时间 7:1:2 切分 → 写 `base_model/data/{train,val,test}_model_input.csv` 与 `data_pre_outputs(7_1_2)/`。

公共工具 `base_model/common/`

- `data_utils.py`: `get_raw_feature_cols` (13 原始输入列)、`prepare_multi_horizon_data` (多步窗口、标准化、`y` 形状 $(N, H \times K)$ horizon-major)、`build_windows_multi_horizon`、`inverse_targets` (反标准化、不 `expm1`) 。
- `train_utils.py`: `train_one_model` (Adam + MSE + 早停)、`predict_scaled_multi`、`make_loader`。
- `metrics_utils.py`: `compute_metrics` → nRMSE/nMAE/NSE/KGE。

基础模型 `base_model/multi_output_model/`

- `models.py`: LSTMFC/LSTMMLP/LSTMKAN/KANRegressor (末层宽度= $H \times K$)、`build_model`、`TARGET_COLS_ORDER`、`HORIZONS`。
- `grid_search_multi_output.py` / `kan_full_grid_search_multi.py`: 对每个架构做坐标下降分步搜索, 按平均 val nRMSE 选优。
- **产物** `grid_search_outputs_multi/`、`kan_full_grid_search_multi_outputs/`: `best_params/best_params_{model}.json` (含 `feature_cols/target_cols/horizons/最优超参/各指标`)、`best_models/best_{model}.pt`、`predictions/{model}/..._best_predictions_{split}.csv` (长表含 horizon)、`metrics/best_metrics_all_models.csv`、`search/*_search_results.csv` (每个试验记录) 。

层次协调 `hierarchical_reconciliation/`

- `reconcile_multi_lstm_kan_best_params.py`: 加载多输出 LSTM-KAN 最优模型, 预测 $(N, 15)$ → 按 horizon 切片 → top-first → 对每个 horizon 做 BU/TD/OLS/WLS/MinT (MinT 协方差仅用验证集残差估计) 。
- **产物** `reconciliation_outputs_multi/`: `summary/reconciliation_metrics.csv` (model×horizon×target×method 指标)、`predictions/reconciled_predictions.csv`。

动态权重端到端协调 `dynamic_weight_end_to_end_reconciliation/`

- `frozen_dynamic_wls_multi.py`: 冻结多输出 LSTM-KAN 基座, 只训练动态权重 MLP; 对每个样本-horizon 的 5 维向量做可微加权最小二乘协调; 保证一致性同时尽量维持/提升精度。
- 产物 `frozen_dynamic_wls_multi_outputs/`: `metrics/..._dynamic_weight_metrics.csv` (Base vs DynamicWLS)、`consistency/..._consistency_by_horizon.csv` (D1 协调前 → D2 协调后, D2≈0 表示精确一致)、`best_params/`、`best_models/`、`search/`。

SHAP SHAP/

- `shap_lstm_kan_multi.py`: 对多输出 LSTM-KAN 的 15 个输出, 逐 (target, horizon) 计算特征级、feature-lag 级 SHAP 与每个特征 SHAP 最大的 lag (供 PDP)。需可用的 shap 环境。

可视化 visualization/ (R)

- 各子目录对应: 网格搜索曲线、多种子箱线图、协调精度、协调对比图、SHAP 图。需 `ggplot2/scales/tidymverse`。注意这些 R 脚本最初按单输出结果编写, 套用多输出新结果需按新列名 (含 horizon) /路径适配。

6. 关键不变量 (改代码时务必遵守)

- **目标列序**: 模型原生顺序 `TARGET_COLS_ORDER = [Green_Algae, Cyanobacteria, Diatoms, Cryptophyta, Algae_Sum]`; 协调要求 top-first `ALL_TARGETS = [Algae_Sum, Green, Cyano, Diatoms, Crypto]`。两者用按名字派生的 `RECON_FROM_MODEL == [4,0,1,2,3]` 桥接 (下游脚本里有断言)。
- **多步 y 列序**: horizon-major, `y[:, h_idx*K + k]` = 第 `HORIZONS[h_idx]` 步、第 `k` 个目标。
- **反变换**: 原始尺度用 `inverse_targets` (只反标准化); 切勿误用旧的 `inverse_log_target` (带 `expm1`, 属单输出 log 管线)。
- **Algae_Sum 只作输出, 绝不作输入。**

7. 已知问题

- `import shap` 在 Bayesian(Py3.13) 崩溃 → SHAP 步骤需换环境运行。
- 含中文路径在某些终端把绝对路径作参数传给 python 会乱码 → 先 `cd` 再用文件名运行。
- `visualization/` 的 R 脚本面向单输出结果, 多输出 (带 horizon) 结果需适配后再用。